NoSQL injections

A login without the password



Reproducing operator injection from Ron et al. (2016)

Five configurations and 150 recorded outcomes

Ercong Kee / Pheobe Neo / Kendrick Poon / Liu Qingyu

A query is a test on a record

DOCUMENT = A STORED RECORD

```
{ username: "alice", password: "…" }
```

Alice’s stored password is a non-empty string in the vulnerable lab.

EQUALITY CONDITION

```
password: "wrong"
```

Does the password equal “wrong”?

OPERATOR CONDITION

```
password: { $ne: "" }
```

Is the password different from an empty string?

Predicate = condition **$ne = not equal** **"" = empty string**

Scope: MongoDB’s document model

MongoDB 7.0 $ne documentation / illustrative teaching record

Where the login goes wrong

ONE CONTINUOUS FALSE-LOGIN PATH

01

Attacker request

Knows “alice” and sends an object

02

App builds query

Copies request values into the filter

03

MongoDB

Alice’s record matches

04

App creates session

Treats the match as password proof

05

Private notes

Protected access is granted

A database match is not password proof.

Ron et al. describe several injection boundaries

OUR MEASURED ROUTE

Operator object → **Changes the query condition** → **A record matches without the correct password**

Operator injection measured in our experiment

OTHER BOUNDARIES IN RON ET AL. (2016)

Query text	JavaScript source	Protocol command	Trusted browser (CSRF)

Experiment focus: the operator route

Ron et al., 2016 / author manuscript and republication

The password value becomes a condition

01 CLIENT REQUEST

username

```
"alice"
```

password

```
{ "$ne": "" }
```

The value is an object.



02 VULNERABLE APPLICATION

```
users.findOne({
  username: body.username,
  password: body.password
})
```

The app lets the request define the password condition.



03 DATABASE

username equals Alice

AND

password is not empty

MATCH

Alice’s stored password is non-empty, so this condition matches.

The parsed type changes the query meaning

OBJECT

```
{ "$ne": "" }
```

MongoDB sees

an operator condition

Alice matches

STRING

```
"{\"$ne\":\"\"}"
```

MongoDB sees

ordinary literal text

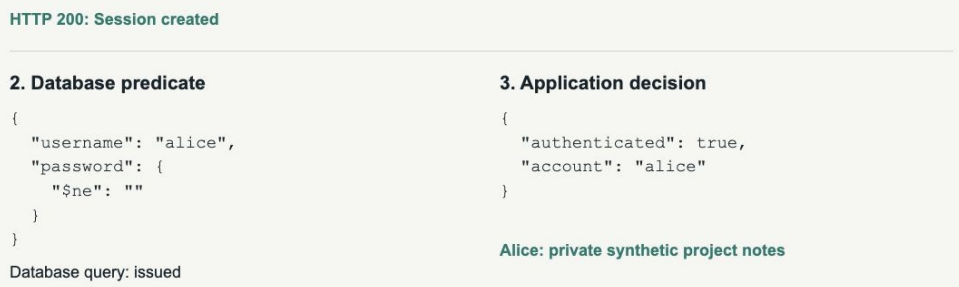
Alice does not match

Same attack, different login decision

ATTACK SUCCEEDS

SESSION	Created
DATABASE	1 query sent
ACCESS	Alice’s notes exposed

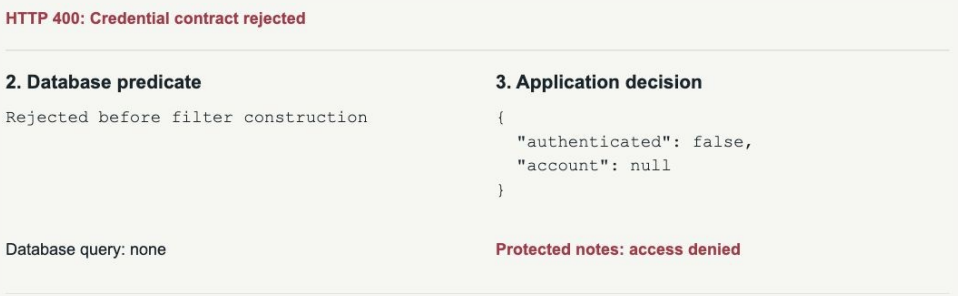
ACTUAL BROWSER CAPTURE



THE SAME ATTACK IS STOPPED

SESSION	Rejected
DATABASE	No query sent
ACCESS	Protected access denied

ACTUAL BROWSER CAPTURE



Which controls stopped the attack?

3 controls still allow the attack

2 application controls stop it before any query

Configuration	Attack outcome	Database query	Correct login
Baseline	**Attack succeeds**	1 query sent	Login works
JavaScript disabled	**Attack succeeds**	1 query sent	Login works
Read-only DB account	**Attack succeeds**	1 query sent	Login works
String validation	**Attack stopped**	No query sent	Login works
Full hardened login	**Attack stopped**	No query sent	Login works

Scripting and read-only controls do not stop this read-only operator attack.

A safe login keeps the password out of the query

SAFE LOGIN PATH

01

Check inputs

Require username and password text

02

Find account

Query by username only

03

Verify password

Check the stored password hash

04

Create session

Only after a successful check

Object password
Rejected. No query.

The password never becomes a database query condition.

Local source / OWASP NoSQL and password-storage guidance

Different injection types need different defences

	Operator injection Measured in our lab	JavaScript injection Literature only
Input form	JSON object with $ne	Text inserted into JavaScript source
Interpreter	MongoDB query engine	JavaScript engine
Effective control	Require expected types Build the query in application code	Never build code from input Disable unused scripting

Only operator injection was demonstrated. JavaScript injection is a literature comparison.

Literature comparison / MongoDB 7.0 $where documentation

What our experiment shows

1 **The attack succeeds** — **Session created**

Operator object logs the attacker in as Alice

2 **The attack is stopped** — **No query sent**

Object rejected before MongoDB receives a query

3 **Legitimate login still works** — **Protected access granted**

Correct credentials retain protected access

Scope: one recorded stack, synthetic accounts and 15 request cases.

Recorded local experiment / results.json and results.csv

The application must keep control of the query

APPLICATION RULE

Choose the query structure.
Validate input values.
Verify the password.

NEXT STUDY

Can flexible search preserve customer separation?

User-selected filters must never remove the required customer filter.



How we tested the controls

15 CASES × **5** CONFIGURATIONS × **2** RUNS = **150** RECORDED RESULTS

FOR EVERY REQUEST, WE RECORDED

Login decision

Did the app create a session?

Protected access

Could the caller open Alice’s notes?

Database query count

Did MongoDB receive a find command?

Separate probes verified that the read-only and scripting controls were active.

Recorded environment

Component	Recorded value
MongoDB	7.0.0
Node.js	v23.6.1
Express / native driver	5.1.0 / 6.20.0
Transport / parser	Loopback HTTP / JSON / express.json
Password verification	scrypt N=131072, r=8, p=1
Evidence	15 cases × 5 modes × 2 runs

Original experiment record. Fresh browser captures: 1 November 2026.

The vulnerable session decision

CLIENT VALUES ENTER THE QUERY

```
const filter = {
  username: req.body.username,
  password: req.body.password
};
const account = await users.findOne(filter);
```

```
if (account) createSession(account);
```

Condensed vulnerable excerpt. A match is incorrectly accepted as password proof.

The hardened session decision

1 Validate exactly two non-empty strings

```
if (!validContract(body)) return reject(400);
```

2 Look up by username only

```
const account = await users.findOne({ username });
```

3 Verify a salted password hash

```
const ok = await verifyPassword(password, storedHash);
```

4 Create a session after verification

```
if (account && ok) createSession(account);
```

Condensed code. Full source includes field/length checks and a dummy hash for unknown users.

Case results C01 to C08

Case	Request shape	Three vulnerable modes	Protected modes
C01	Valid credentials	Login succeeds	Login succeeds
C02	Wrong string password	Login rejected	Login rejected
C03	Unknown username	Login rejected	Login rejected
C04	Password operator object	Attack succeeds	Stopped before query
C05	Username operator with wrong password	Login rejected	Stopped before query
C06	Operator-looking literal string	Login rejected	Login rejected
C07	Null password	Login rejected	Stopped before query
C08	Array password	Login rejected	Stopped before query

Vulnerable modes: baseline, scripting disabled and read-only. Both runs agree.

Case results C09 to C15

Case	Request shape	Three vulnerable modes	Protected modes
C09	Missing password	Login rejected	Stopped before query
C10	Numeric password	Login rejected	Stopped before query
C11	Extra field with valid credentials	Request accepted	Stopped before query
C12	Top-level array	Login rejected	Stopped before query
C13	Both fields contain operators	Attack succeeds	Stopped before query
C14	Empty password	Login rejected	Stopped before query
C15	Empty username	Login rejected	Stopped before query

Vulnerable modes: baseline, scripting disabled and read-only. Both runs agree.

The controls were independently checked

Control	Independent observation
Read-only identity	Authenticated as labreader with the read role
Read permitted	Alice lookup succeeds
Write denied	MongoDB blocks the write (error 13)
JavaScript probe enabled	The fixed $where probe returns Alice
JavaScript probe disabled	The same probe fails as expected (error 6108304)

Both controls are active, but neither stops attack C04.

The fixed $where expression tests configuration, not attacker-controlled JavaScript.

Captured evidence: attack succeeds

```
{"username":"alice","password":{"$ne":""}}
```

HTTP 200: Session created

2. Database predicate

```
{
  "username": "alice",
  "password": {
    "$ne": ""
  }
}
```

Database query: issued

3. Application decision

```
{
  "authenticated": true,
  "account": "alice"
}
```

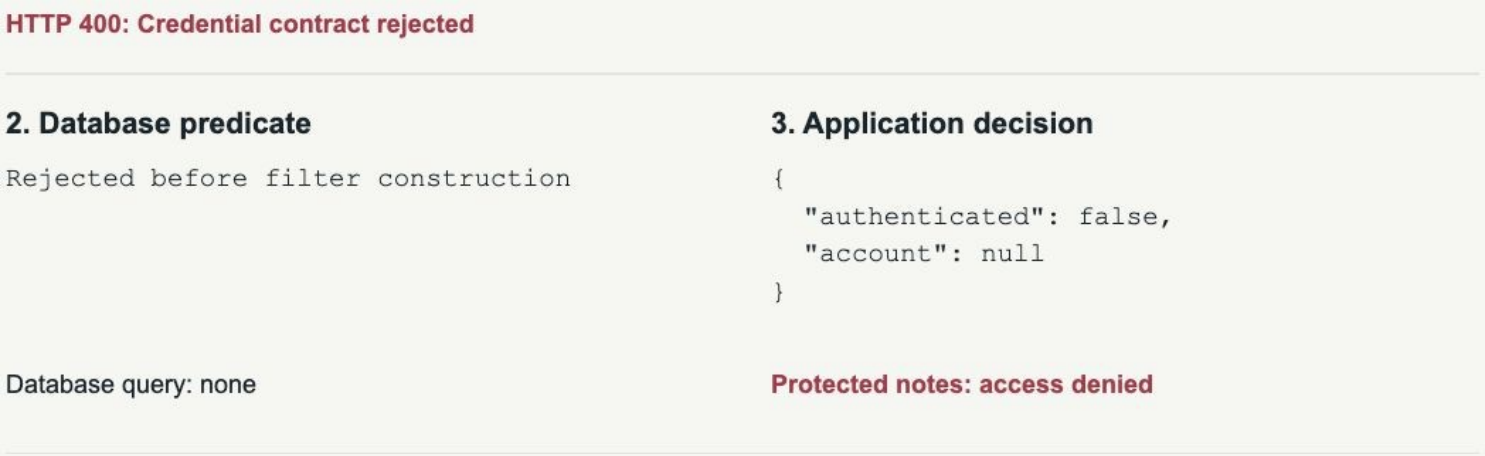
Alice: private synthetic project notes

HTTP		
Session created		200
DATABASE		
Query sent		1 find
ACCESS		
Alice’s notes exposed		granted

The vulnerable app treats a record match as password proof.

Captured evidence: attack stopped

```
{"username":"alice","password":{"$ne":""}}
```



HTTP

Credential contract rejected — 400

DATABASE

No query sent — 0 find

ACCESS

Protected notes denied — denied

The hardened app rejects the object before constructing a filter.

What remains untested

ESTABLISHED

1 Operator payload creates a session

2 Across 150 recorded outcomes, both runs agreed in all five configurations

3 The fix rejects before query and retains correct login

NOT ESTABLISHED

1 No JavaScript exploit or prevalence estimate

2 Not all drivers or database versions

3 No administrator-access or production-security claim

Next experiment: allow flexible search without leaking another customer’s records.

Article and database references

Ron, Shulman-Peleg and Puzanov (2016)
Analysis and Mitigation of NoSQL Injections
IEEE Security & Privacy 14(2), 30–39

DOI: 10.1109/MSP.2016.36

Author-uploaded manuscript and authors' InfoQ republication
Original classes: tautologies, union queries, JavaScript injection, piggybacked queries and origin violation.

MongoDB 7.0: $ne and $where documentation
Full URLs in the notes and team guide.

Defences and reproducible evidence

OWASP NoSQL Security Cheat Sheet
OWASP Password Storage Cheat Sheet

github.com/Qingyu255/CS440-Project

Evidence commit: c8252d2
Source and results are fixed at this revision.



results.json / results.csv / server.mjs / experiment.mjs
Fresh captures and team guide accompany the revised deck.